DocuMind: In-Depth Technical Architecture & Overview

DocuMind is an advanced, enterprise-grade Retrieval-Augmented Generation (RAG) platform. Unlike standard RAG systems that blindly return answers, DocuMind focuses heavily on **Uncertainty Quantification (UQ)**, **Risk Control**, and **Retrieval Robustness**.

This document breaks down every granular technical detail, algorithm, and flow implemented in this project.

---

1. The Core Problem: LLM Hallucinations & Overconfidence

Standard LLMs suffer from *saturation*—they often output highly confident scores even when hallucinating. They cannot distinguish between:

1. **Aleatoric Uncertainty:** Uncertainty due to noisy or conflicting context (e.g., the document contradicts itself).
2. **Epistemic Uncertainty:** Uncertainty due to lack of knowledge (e.g., the document doesn't contain the answer, and the model doesn't know it).

DocuMind solves this using a multi-channel uncertainty estimation pipeline consisting of Semantic Entropy, Evidential Deep Learning (EDL), and Conformal Prediction.

---

2. Component Breakdown

A. Corrective Retrieval Pipeline (`corrective.py`)

Standard dense retrieval often misses context if the user's query is poorly phrased.

1. **Query Expansion:** The system uses an LLM (Groq) to expand the original query into N (default: 3) distinct, semantically related queries.
2. **Parallel Dense Retrieval:** It searches the vector database (ChromaDB) for all N queries simultaneously, fetching a large pool of candidate chunks.
3. **Neural Reranking:** Because retrieving too many chunks dilutes the context, DocuMind uses **Cohere's Neural Reranker** (`rerank-english-v3.0`). It computes a cross-attention relevance score between the query and every candidate chunk, returning only the highly relevant top K (default: 8) chunks.

B. Semantic Entropy (`Confidence.py`)

Instead of asking the LLM "how confident are you?", Semantic Entropy measures uncertainty by evaluating the *consistency of meaning* across multiple stochastic generations.

1. **Stochastic Sampling:** The LLM generates M different answers at a higher temperature.
2. **Semantic Clustering:** The system extracts the embeddings of all M answers and clusters them based on cosine similarity. Answers that mean the same thing (e.g., "The capital is Paris" and "Paris is the capital") fall into the same **meaning cluster**.
3. **Entropy Calculation:** It calculates the Shannon entropy over these meaning clusters.
    - *High Entropy:* The model generated many conflicting answers (High Epistemic Uncertainty / Hallucination risk).
    - *Low Entropy:* The model generated semantically equivalent answers every time (High Confidence).

C. Evidential Deep Learning (EDL) Fusion (`evidential.py`)

In standard systems, different signals (retrieval score, self-consistency, semantic entropy) are fused using simple linear weights. DocuMind uses **Subjective Logic and Dirichlet Distributions**.

1. **Evidence Collection:** Signals are converted into "evidence" ($e > 0$).
2. **Dirichlet Parameterization:** The evidence parameterizes a Dirichlet distribution: $\alpha = e + 1$. The total evidence is $S = \sum \alpha$.
3. **Belief Masses:** The system calculates the belief mass for the answer being Correct (b_c) and Incorrect (b_i).
4. **Vacuity (Epistemic Uncertainty):** Crucially, EDL calculates **Vacuity** ($u = 2 / S$). If the total evidence is low, vacuity is high, meaning the system mathematically admits "I don't have enough evidence to make a confident prediction."

D. Split Conformal Prediction (`conformal.py`)

Static confidence thresholds (e.g., "reject if score < 0.5") are brittle. They fail when the data distribution shifts. Conformal Prediction provides **statistically guaranteed risk control**.

1. **Calibration Set:** As users provide feedback (or via synthetic evaluation), the system builds a calibration set of *non-conformity scores* ($1 - \text{fused_confidence}$).
2. **Empirical Quantiles:** Given a user-defined error tolerance α (e.g., $\alpha = 0.1$ for 90% accuracy guarantee), the system calculates the exact quantile threshold $\hat{q}$ over the calibration set.
3. **Dynamic Abstention:** During inference, if the model's uncertainty exceeds $\hat{q}$, the system gracefully **abstains** (escalates to a human). This mathematically guarantees that the system will maintain its target coverage and bound hallucination rates, regardless of domain shifts.

---

3. The Data Flow: Step-by-Step

When a user submits a query via the Frontend UI, here is exactly what happens:

1. **Upload & Ingestion (`/documents/upload`):**
    - PyMuPDF extracts text from PDFs.
    - Text is chunked (overlapping sliding windows).
    - Chunks are embedded using an embedding model and stored in ChromaDB.
2. **User Query (`/query/uncertainty`):**
    - The query hits the FastAPI router.
    - **Corrective Retrieval:** Query is expanded $\to$ ChromaDB queried $\to$ Results reranked via Cohere.
    - **Answer Generation:** Groq LLM generates the primary RAG answer using the top reranked context.
3. **Uncertainty Quantification:**
    - Groq LLM is sampled M times.
    - Semantic Entropy clusters the answers and computes cluster probabilities.
    - Retrieval similarity mean, Self-Consistency, and Semantic Confidence are calculated.
    - The **EDL Fusion Engine** converts these signals into Dirichlet belief masses ($b_{correct}, b_{incorrect}$) and Vacuity.
    - A final `predictive_confidence` score is extracted from the Dirichlet expected probability.

4. **Conformal Risk Check:**
    - The `predictive_confidence` is checked against the dynamically calculated Conformal Threshold $\hat{q}$.
    - If confidence $< 1 - \hat{q}$, the backend flags the answer as an **Escalation / Abstention**.
5. **Frontend Rendering:**
    - The React UI displays the Chat message.
    - A highly detailed **Confidence Card** allows the user to expand and inspect:
        - The exact number of Meaning Clusters.
        - The exact Semantic, Epistemic, and Aleatoric uncertainty bars.
        - The EDL Dirichlet Strength and Vacuity.
        - The current Conformal Calibration status (Active vs Bootstrapping).
6. **Feedback Loop (`/query/feedback`):**
    - If a human reviews an answer, they can submit feedback (Correct/Incorrect).
    - This feedback updates the Conformal Predictor's calibration store, dynamically tightening or loosening the abstention threshold in real-time to maintain statistical guarantees.

---

4. Evaluation & Metrics (`evaluate_edl.py`)

To prove the UQ pipeline works, the system includes a rigorous evaluation script. It generates synthetic data simulating RAG outputs and computes:

- **ECE (Expected Calibration Error):** Measures how closely predicted confidence aligns with actual accuracy. EDL consistently achieves lower ECE than linear fusion.
- **Brier Score / NLL:** Measures the accuracy of probabilistic predictions.
- **Selective Risk / AUROC:** Proves that as the conformal abstention threshold increases, the risk on the *remaining* (non-abstained) queries drops monotonically.

Conclusion

DocuMind is not just another wrapper around an LLM. It is a highly robust, mathematically grounded architecture designed for high-stakes environments (legal, medical, financial) where **knowing when to say "I don't know" is just as important as generating the correct answer**.